Math Manufacturing

On the Civilizational-Scale Impact of Kernel-Verified Mathematics as a Production System

Sean Chatman

Abstract

This thesis examines a single empirical result — the routing of a 219-statement mathematics corpus through automated Lean 4 kernel verification, yielding a disjoint, receipted partition of 179 verified, 4 rejected, 18 dependency-blocked, and 17 deduplicated statements — as an instance of an existing formal pattern already present in this project's prior work: the Chatman Equation's manufacturing morphism $\mathcal{A} = \mu(\mathcal{O}^*)$. We argue that this pipeline is not a new theoretical apparatus but a second working instantiation of a pattern already implemented once (an RDF-to-PDDL planning compiler). Part I (Chapters 2–4) is evidence-bound: every quantitative claim is derived live from artifacts on disk and cited to the exact query or file that produced it. Part II (Chapter 5) is explicitly speculative, marked throughout by a dedicated *Speculation* environment, and addresses what changes at civilizational scale if verified mathematics acquires a manufacturing-like cost curve. Chapters 6–8 situate the work against prior autoformalization efforts and state its limitations without euphemism.

Contents

Chapter 1

Introduction

1.1 Thesis statement

A mathematics corpus of 219 statement instances, drawn from seven source papers of an ongoing thesis series, was passed through an automated pipeline that lowers each statement into Lean 4 syntax and submits it to the Lean kernel for admission or rejection. The result is a disjoint partition: 179 statements kernel-verified, 4 kernel-rejected after bounded retry, 18 correctly withheld because they transitively depend on an out-of-scope statement, and 17 cross-paper restatements collapsed by shared label before scheduling. Every number in that sentence is derived live in Chapter 4 from artifacts on disk, not asserted from memory.

This thesis makes one narrow technical claim and one broad, explicitly labeled speculative claim, and it does not let the two blend.

The narrow claim (Part I, Chapters 2–4): this pipeline is not a new theoretical apparatus. It is a second working instance of a pattern already present in this codebase — the *manufacturing morphism* $\mathcal{A} = \mu(\mathcal{O}^*)$ that the project's own prior work already names and already implements once, for a different domain (RDF ontology → PDDL planning text). Naming the Lean pipeline "math manufacturing" is an act of recognition, not invention: the same admit/refuse/receipt structure recurs, and Chapter 3 makes the correspondence explicit, term for term.

The broad claim (Part II, Chapter 5): if verified-mathematics production can be organized as a pipeline with a defect taxonomy, a throughput number, and a receipt for every unit produced — the way physical manufacturing is — then something changes about the economics of formal verification, mathematical research, and mathematics education at a civilizational scale. This claim is speculative. It is not derived from the 219-statement result; it is motivated by it. Chapter 5 marks every claim of this kind with a dedicated SPECULATION environment so that a reader — or an adversarial reviewer — can mechanically distinguish "this is what we built and verified" from "this is what we think it might mean if it scales."

1.2 Why the distinction matters

A prior draft of this argument, written as prose in a conversational exchange rather than as a checked document, contained a real accounting error: a partition of 219 statements into verified/rejected/blocked/ deduplicated categories that summed to 218, not 219, because two originally-verified pilot statements had been silently dropped from a consolidated receipts file during a merge step. The error was caught, traced to its exact cause, and fixed by re-verifying the two missing

statements against the real Lean kernel and re-inserting their receipts — documented in full in Section 4.5. That episode is retained in this thesis, not smoothed over, because it is itself evidence for the thesis statement: a manufacturing discipline is distinguished from a narrative one precisely by the fact that its errors are mechanically detectable and its claims are falsifiable against artifacts on disk. A thesis arguing for that discipline should be held to it.

1.3 Structure of this document

Chapter 2 establishes the pre-existing formal machinery this thesis builds on: the Chatman Equation, admission algebra, receipts, and the first working manufacturing morphism (`src/mfg.rs`). Chapter 3 states the second instantiation — math manufacturing — as a formal correspondence to the first. Chapter 4 reports the empirical run: exact counts, the two infrastructure bugs found and fixed during the run, and the receipts-consolidation gap and its fix. Chapter 5 is the speculative civilizational-impact argument. Chapter 6 situates this work against prior autoformalization efforts. Chapter 7 states limitations without euphemism. Chapter 8 concludes.

Chapter 2

Background: the Chatman Equation and manufacturing morphisms

This chapter is entirely evidence-bound: every construct below is quoted or paraphrased from artifacts already present in this codebase prior to this thesis, cited by exact file path.

2.1 The Chatman Equation

The project's foundational equation, stated in `ORIGINAL_REQUEST.md` and the admission-algebra paper (`docs/thesis/01_admission_algebra.tex`), is

$$\mathcal{A} = \mu(\mathcal{O}^*), \qquad \mathcal{O} \xrightarrow{\alpha} \mathcal{O}^* \xrightarrow{\mu} \mathcal{A} \xrightarrow{\text{receipt}} \mathcal{R}.$$

Reading left to right: a raw observation space $\mathcal{O}$ is passed through an *admission map* α into an admitted space $\mathcal{O}^*$ (only observations that pass admission continue); the *manufacturing morphism* μ maps admitted observations to actuation $\mathcal{A}$; and every actuation is paired with a *receipt* $\mathcal{R}$ — a computed, content-addressed record of what happened, never asserted-in by fiat. μ is named, in the project's own vocabulary, the "manufacturing morphism." This thesis's title is a direct restatement of that name applied to a new domain.

2.2 Admission algebra: refusal as data

`01_admission_algebra.tex` establishes that admission cannot be decided by meaning alone (a Rice's-theorem-style undecidability result, `thm:rice` in the corpus) and instead formalizes admission as an algebra over a refusal monoid: refusal is data, not a side effect (`ax:refusal`), with a bottom element $\bot$ characterizing full admission (`prop:bottom`) and monotonicity under composition. The practical corollary, realized in code, is that every admission stage either produces a value or a named, typed refusal — never a panic, never a silent default. This is the same discipline this project's own `CLAUDE.md` states as an invariant: "every error is a typed `Refusal` variant."

2.3 Receipts: computed, not asserted

`02_receipt_cryptography.tex` formalizes the receipt chain: a BLAKE3 content hash folded against a genesis value, so that a receipt for step n is a function of the receipt for step $n-1$ and the actuation at step n, never independently fabricated. This is the same mechanism used throughout

this thesis's own evidence: every count in Chapter 4 is derived by re-running a query against a file on disk, not copied from a prior claim.

2.4 The first manufacturing morphism: `src/mfg.rs`

A concrete implementation of μ already exists in this codebase, independent of the work in this thesis. `src/mfg.rs` is a literal manufacturing pipeline: it loads a Turtle RDF ontology describing planning domain facts (types, predicates, actions, a problem instance), extracts a STRIPS8-profile intermediate representation via SPARQL, enforces PDDL8 structural bounds (arity, conjuncts, and parameter counts each ≤ 8, mirroring `wasm4pm_compat::pddl`'s `PDDL8_MAX_*` constants) as a hard admission gate, and emits PDDL domain and problem text as its actuation. Its module doc comment states the shape directly: "RDF ontology → PDDL8 domain/problem text."

Its error type,

```
enum MfgError { Graph(String), BoundExceeded { what, limit, got, detail },
Shape(String), Pddl8(String) }
```

is a defect taxonomy for this pipeline: a graph-parse failure, a structural-bound violation (with the exact limit and the exact value that exceeded it), a missing or malformed ontology fact, or a round-trip failure back through the downstream planner. Every one of these is a typed refusal in the admission-algebra sense of Section 1.2 above — none of them is a panic or a silent fallback.

Definition 2.1 (μ-pipeline)**.** A *μ-pipeline* is a concrete instantiation of $\mathcal{A} = \mu(\mathcal{O}^*)$ consisting of: (i) a raw input domain $\mathcal{O}$; (ii) an admission map $\alpha : \mathcal{O} \to \mathcal{O}^* \cup \{\bot\} \times \mathcal{R}_f$ that either admits an input or produces a typed refusal; (iii) a manufacturing morphism $\mu : \mathcal{O}^* \to \mathcal{A}$ producing a concrete actuation artifact; and (iv) a receipt function computing $\mathcal{R}$ from the actuation, never asserted.

`src/mfg.rs` instantiates Definition 2.1 with $\mathcal{O}$ = RDF ontology text, α = SPARQL extraction plus PDDL8 bound enforcement, μ = domain/problem text emission, and $\mathcal{A}$ = PDDL8 text validated by round-trip through `bcinr-pddl`. Call this **μ-pipeline 1**. Chapter 3 instantiates the same definition a second time, with mathematics statements as the input domain and the Lean 4 kernel as the admission map — **μ-pipeline 2** — and states the correspondence between the two, term for term.

Chapter 3

Math manufacturing: the second manufacturing morphism

3.1 The correspondence, term for term

Table 3.1 states the correspondence between μ-pipeline 1 (`src/mfg.rs`, Section 2.4) and μ-pipeline 2, the autoformalization pipeline built and run this thesis reports on.

Term	**μ-pipeline 1 (RDF $\rightarrow$ PDDL)**	**μ-pipeline 2 (Math $\rightarrow$ Lea**
$\mathcal{O}$ (raw input)	Turtle ontology text	LaTeX statement text, corpus
α (admission map)	SPARQL extraction + PDDL8 bounds	Lean 4 kernel typecheck
$\mathcal{O}^*$ (admitted domain)	STRIPS8 intermediate representation	Lean 4 syntax draft (pre-kern
μ (manufacturing morphism)	domain/problem text emission	`.lean` source file emission
$\mathcal{A}$ (actuation)	PDDL8 text, round-tripped	kernel-accepted `.lean` artifac
Defect taxonomy	`MfgError` (Graph/BoundExceeded/Shape/Pddl8)	verified / unformalized / bloc
$\mathcal{R}$ (receipt)	(implicit: round-trip check)	`formalization_receipts.js`

Table 3.1: Term-for-term correspondence between the two working μ-pipeline instantiations in this codebase.

The correspondence is not decorative. Both pipelines share the structural property that admission is a hard, typed gate rather than a best-effort check: μ-pipeline 1 will not emit PDDL8 text that violates its arity/conjunct/parameter bounds, and μ-pipeline 2 will not record a statement as `verified` without an actual, successful invocation of the real `lean` binary against it (Section 3.2.1).

3.2 The pipeline in detail

Definition 3.1 (Math manufacturing pipeline)**.** Given a corpus of statement individuals $s \in \mathcal{O}$, each carrying a LaTeX statement string, a `kind` (axiom, definition, construction, theorem, proposition, corollary, or lemma), and a set of `dependsOn` edges to other statements:

1. **Scoping.** Statements sharing a label with an already-processed statement are deduplicated to the first occurrence (Section 3.2.2); a designated out-of-scope statement and its transitive dependents are excluded a priori (Section 3.2.3).
2. **Scheduling.** The remaining statements are ordered into topological waves over the `dependsOn` DAG (Kahn's algorithm): wave 0 contains statements with no unresolved dependencies; wave n

contains statements whose dependencies were all resolved (verified or blocked) by the end of wave $n-1$.

3. **Admission.** Within a wave, each statement is drafted into Lean 4 syntax by an agent given the statement's LaTeX text and the already-verified Lean declarations of its dependencies, then typechecked by the real `lean` kernel. On rejection, the exact kernel error is fed back to the same agent for revision, bounded to three attempts total.
4. **Disposition.** A statement is recorded `verified` on kernel acceptance, `unformalized` after three failed attempts (with the last real kernel error retained), or `blocked` if any dependency did not resolve to `verified`.
5. **Receipt.** Every disposition is appended as one line to `formalization_receipts.jsonl`, regardless of outcome.

3.2.1 No self-reported success

A structural requirement of this pipeline, inherited from the admission- algebra discipline of Section 2.2, is that an agent's own claim of success is never trusted as the disposition. Every `verified` disposition in Chapter 4 is independently re-checked by invoking the real `lean` kernel directly against the `.lean` file on disk, outside of and after the agent's own report. This check caught one concrete discrepancy during the run reported in Chapter 4: a statement self-reported as `unformalized` was found, on independent re-verification, to type-check cleanly, and its disposition was corrected before being recorded in the receipts file.

3.2.2 Deduplication

The source corpus contains 219 statement individuals but only 202 distinct labels (Section 4.1): 17 labels are restated across more than one source paper (the same concept, independently re-derived in each paper's own argument). Because `dependsOn` edges reference labels rather than per-paper individuals, the scheduler resolves each label once, at its first occurrence, rather than re-formalizing 17 statements that would produce identical Lean declarations.

3.2.3 A priori exclusion

One statement, `thm:faithful` (a faithfulness/projection theorem with cryptographic and probabilistic content), is excluded from scope before scheduling begins, together with every statement that transitively depends on it. This is not a failure of the pipeline; it is a scoping decision reported honestly, discussed further in Chapter 7: bare Lean 4 core, with no mathlib installed, is not the appropriate kernel for cryptographic/probabilistic content, and formalizing it would require a different toolchain (Chapter 7 returns to this point).

Chapter 4

Empirical results

Every number in this chapter was re-derived at drafting time, not copied from earlier conversational prose, by two commands: a Python pass counting the `status` field across every line of `tools/paper-factory/lean-pilot/formalization_receipts.jsonl`, and a SPARQL query against `docs/thesis/rdf/corpus.ttl` selecting every `math:Statement` individual's label and kind. Both are reproducible by any reader with the corpus and receipts file in hand.

4.1 Corpus numbers

Quantity	**Count**
Statement individuals (IRIs) in corpus	219
Distinct statement labels	202
Duplicate-label IRIs (restated across papers, deduplicated)	17
Proof individuals	95
Distinct symbols (post notation-conflict merge)	53

Table 4.1: Corpus-level counts, re-derived by SPARQL against `docs/thesis/rdf/corpus.ttl`.

4.2 Formalization disposition

Disposition	**Count**
Verified (kernel-accepted)	179
Unformalized (3 attempts exhausted)	4
Blocked (dependency not verified)	18
Excluded a priori (`thm:faithful` + n/a)	1
Total distinct labels	202

Table 4.2: Disposition of all 202 distinct statement labels, re-derived from `formalization_receipts.jsonl` (201 lines) plus the one a-priori exclusion, which by design receives no receipt line.

$179 + 4 + 18 + 1 = 202$, and $202 + 17$ (deduplicated IRIs) $= 219$, the full corpus IRI count. This is a disjoint partition with no remainder category.

Of the 202 distinct labels, 7 were already formalized and verified prior to the wave-scheduled run reported below, as a manual pilot proving the reject-fix-retry loop worked at all: `ax:obs`, `def:denialabstract`, `thm:rice`, `def:adm`, `def:mu`, `def:receipt`, `prop:monoid`. The remaining 194 were processed by the automated wave scheduler described in Chapter 3, in 8 topological waves.

4.3 The four kernel rejections

Four statements exhausted three real `lean` invocations without kernel acceptance. Their kind and the terminal error are retained verbatim in the receipts file, not paraphrased into success: `thm:mrrsep` (theorem; 4 kernel errors on the final attempt), `prop:bottom` (proposition; `Function expected` at a specific line), `prop:cone` (proposition; a type mismatch), `cor:allstages` (corollary; a type mismatch on a compound boolean term). These are reported as genuine limits of the bare-Lean-core, bounded-retry approach (Chapter 7), not as pipeline defects.

4.4 Two infrastructure bugs, found and fixed during the run

The wave-scheduler harness itself failed twice during development, in ways worth reporting as evidence that the pipeline's own correctness was checked rather than assumed.

Args-as-string. The workflow orchestration tool's top-level data-passing parameter arrives as a raw JSON string rather than a parsed value in its scripting sandbox. An early version of the harness attempted to pass the full corpus (including LaTeX statement text) through this channel; large payloads were corrupted in transit. The fix was structural, not a patch: only a small label/kind/dependency skeleton is passed through the orchestrator, and each per-statement formalization agent fetches its own statement's LaTeX text directly via a SPARQL query it runs itself.

Wave-scheduler precompute bug. An early version computed every topological wave upfront from a results map that was never updated with live outcomes, so after wave 0 genuinely ran, the scheduler evaluated wave 1's readiness against stale (empty) results and force-blocked all remaining statements. The fix interleaves computation and execution: wave n's readiness is computed from the live, continuously updated results map immediately before wave n executes, not from a value computed once at the start. The corrected run was resumed from cache (the tool's `resumeFromRunId` mechanism), so the 54 already-valid wave-0 results were reused rather than recomputed.

4.5 The receipts-consolidation gap

After the wave-scheduled run completed, the consolidated `formalization_receipts.jsonl` was found to contain 199 lines, not the expected 201 (7 pilot + 194 processed): the merge step that combined the manual-pilot receipts with the wave-run receipts had silently dropped the standalone entries for `def:receipt` and `prop:monoid`, even though both statements' `.lean` files were verified and present on disk. This was caught by the same independent-verification discipline stated in Section 3.2.1: re-running the real `lean` kernel directly against both files confirmed both type-check (the `elan`-managed `lean` binary invoked directly on `def_receipt.lean` and `prop_monoid.lean`, both exit 0), and the two missing receipt lines were re-added, bringing the file to 201 lines and the counts in this chapter to their final, reconciled values.

Finding 4.1. A consolidation step (merging two receipt sources into one file) is itself a point where evidence can be silently lost even when the underlying artifacts remain correct on disk. The fix here was procedural (independent kernel re-verification of every file, not just trusting the consolidated file's line count) rather than a change to the pipeline's formalization logic.

Chapter 5

Civilizational-impact analysis

Every claim in this chapter is marked with the SPECULATION environment defined in the preamble. None of it is derived from the 219-statement result in the sense that Chapters 3–4 are derived: it is motivated by that result, and it is falsifiable in principle, but it is not kernel-checked the way a Lean statement is. Chapter 8's verification step confirms mechanically that no claim in this chapter escapes this marking.

5.1 An anchor number, not an estimate

Before speculating, one real number, taken directly from git commit timestamps, not from memory or narrative: the commit that added the paper-factory tooling and compiled a validated 41-page mega-paper PDF (`55e4cf4`, 2026-07-05 01:27:30) and the commit recording 172 newly kernel-verified Lean statements across an 8-wave scheduled run (`8d15996`, 2026-07-05 02:26:31) are 59 minutes apart. In that interval: 13 groups of genuine shared mathematical notation were merged and 2 accidental symbol collisions were resolved at the source; the whole corpus was re-validated (SHACL, dangling-citation, notation- conflict); and a topologically-scheduled, kernel-verified formalization run produced 172 new `.lean` artifacts, each independently re-checked against the real Lean kernel.

Speculation 5.1. A domain expert manually selecting, drafting, and Lean-checking a single nontrivial formalization — reading the source statement, choosing encodings for its dependencies, writing tactics, iterating against kernel errors — rarely takes less than thirty minutes even for a routine lemma, and can take hours for a genuinely novel argument. Taking thirty minutes as a deliberately low floor, 172 statements implies at least 86 expert-hours of equivalent manual effort, compressed into a 59-minute wall-clock window. This is not a claim that the automated result is equivalent in depth or reliability to 86 hours of expert attention per statement — Chapter 7 states plainly where the automated pipeline's guarantees stop — it is a claim that the throughput *ratio*, on this one data point, is large enough that "how many PhDs would this replace" is the right order-of-magnitude question to be asking, rather than an obviously rhetorical one.

5.2 What changes if verified math has a manufacturing cost curve

Speculation 5.2. Physical manufacturing did not merely make existing artisanal goods cheaper; it changed which goods were worth producing at all, because the marginal cost of the tenth unit stopped resembling the marginal cost of the first. If kernel-verified mathematical statements can be produced at a cost curve that similarly flattens with corpus size — because each newly verified statement enlarges the reusable, already-proved vocabulary available to every subsequent

statement's formalization attempt, exactly as Chapter 3's wave scheduler does — then the population of mathematical claims worth formally verifying stops being limited to results whose importance justifies a human expert's dedicated attention, and starts including large swaths of routine, load-bearing lemmas that are currently left as "clearly true" prose because no one has hours to spend proving them formally.

Speculation 5.3. For mathematical-research throughput, this suggests a shift in what a research paper's "proof" section is for: not the sole evidence of correctness, but a draft that a math-manufacturing pipeline lowers into a kernel-checked artifact as a matter of course, the way a compiler lowers source to machine code today. The reviewer's question moves from "is this proof convincing" to "does this proof compile" for the mechanical portions, freeing expert attention for the genuinely novel steps — plausibly the four kernel-rejected statements in this thesis's own run (Section 4.1) are exactly the kind of step that should keep demanding a human.

Speculation 5.4. For formal-verification adoption, the barrier has historically been that formalizing existing mathematics is itself a research-grade skill, so verified libraries (Mathlib and its counterparts) grow only as fast as a comparatively small community of specialists can push them forward. A pipeline of the shape in Chapter 3 — LLM drafting, kernel admission, automated reject-fix-retry, a compounding verified- vocabulary library — targets exactly that bottleneck. It does not remove the need for the specialist community; it changes what fraction of a specialist's day goes to formalizing routine statements versus extending the frontier.

Speculation 5.5. For mathematics education, a manufacturing-style pipeline that can formalize a routine claim in minutes suggests a different kind of exercise: a student's proof attempt checked by the same kernel used here, with the same reject-fix-retry loop turned into an interactive tutor rather than an autonomous batch process. This is a natural extension, not a novel proposal — interactive theorem provers have been used pedagogically before — but the wave-scheduled, dependency- aware version in this thesis suggests the tutoring loop could scale to a whole curriculum's dependency graph rather than one exercise at a time.

5.3 The load-bearing caveat

Speculation 5.6. Every claim above assumes the throughput advantage observed on this one 219-statement corpus, run against bare Lean 4 core with no mathlib, generalizes to larger, more mathematically diverse corpora and to libraries with an existing large body of dependencies to search and reuse. That assumption is untested by this thesis and is the natural target of the future work named in Chapter 7.

Chapter 6

Related work

This chapter situates the empirical result of Chapter 4 against prior autoformalization and LLM-assisted theorem-proving work. It does not claim priority for the general idea of using a language model to draft formal proofs and a kernel to check them — that idea predates this thesis by years — it claims a specific, narrower thing: a wave-scheduled, dependency-aware harness with automated reject-fix- retry, applied to a from-scratch bare-Lean-core corpus with no mathlib dependency, is a comparatively underexplored point in the design space.

6.1 Large-scale verified libraries

Mathlib (Lean 4) and comparable libraries for Coq and Isabelle represent decades of cumulative, mostly human-driven formalization effort, and are the existing evidence that a compounding verified-vocabulary library is possible at scale — the same structural property Chapter 3's wave scheduler exploits at a much smaller size (a 194-statement scope, not a library of hundreds of thousands of results). This thesis's corpus deliberately does not depend on Mathlib; every formalization in `tools/paper-factory/lean-pilot/` is proved from bare Lean 4 core. That is a scope limitation (Chapter 7), not a claim that avoiding Mathlib is preferable in general.

6.2 LLM-assisted formal proof search

Systems that pair a large language model with an interactive theorem prover's kernel to search for and check proofs — including competition-mathematics-focused systems reported to reach medal-level performance on olympiad-style geometry and algebra problems — share this thesis's core loop: draft, check, revise on kernel rejection. The distinguishing features of the pipeline in Chapter 3 are (a) the input is a pre-existing, dependency-linked corpus of a research paper series' own definitions and theorems, rather than a benchmark of independent competition problems, and (b) scheduling is explicitly topological over a real `dependsOn` graph, so that a formalization's available vocabulary strictly grows with each completed wave, rather than each attempt starting from a fixed background library.

6.3 Autoformalization of existing mathematical text

Prior work translating existing informal mathematical text (textbooks, papers) into formal statements and proofs establishes that the informal-to-formal lowering step itself is a nontrivial research problem,

independent of proof search. This thesis's corpus sidesteps part of that problem by construction: the source statements were already extracted into structured RDF individuals with an explicit `kind` and `dependsOn` graph (the paper-factory pipeline, predating this thesis), rather than starting from free-form prose. The informal-to-RDF step is therefore prior work of this project's own, not addressed fresh here, and is itself a limitation acknowledged in Chapter 7.

6.4 What is not addressed

This thesis does not attempt cross-kernel verification (checking the same statement in more than one proof assistant, e.g. Lean and Coq independently, for redundancy against a single kernel's own defects), net-new theorem generation (proposing statements not already present in the source corpus), or benchmarking against a standard autoformalization dataset. Each is a natural extension of the harness in Chapter 3, not attempted here for scope reasons stated in Chapter 7.

Chapter 7

Limitations

Stated without euphemism, in the order they matter most.

7.1 Bare Lean core, no mathlib

Every formalization in this thesis's corpus is checked against Lean 4 core with no mathlib installed. This bounds the pipeline to statements whose vocabulary can be built from scratch in a reasonable number of lines — exactly why `thm:faithful` (cryptographic and probabilistic content) was excluded a priori rather than attempted: bare core has no probability or cryptographic-primitive library to build on, and formalizing one from scratch was out of scope. This is a genuine capability boundary, not a stylistic choice: a corpus leaning further into analysis, probability, or algebraic structures with deep existing mathlib support would need a different toolchain to get a fair test of the wave-scheduling and reject-fix-retry mechanisms this thesis actually contributes.

7.2 Single kernel, no cross-check

Every statement is checked against exactly one proof assistant's kernel. A kernel defect (a soundness bug in Lean 4 itself) would not be caught by this pipeline, since there is no independent second kernel (Coq, Isabelle) checking the same statement. This is a real, if low-probability, gap: production-grade formal-verification claims at civilizational scale (Chapter 5) would reasonably want cross-kernel redundancy for anything load-bearing.

7.3 Scope: 194 of 219, not the whole corpus

The wave-scheduled run covers 194 of 219 statement instances (202 unique labels minus the 7 already-verified pilot statements and 1 a-priori exclusion, doubled back through Chapter 4's accounting). The remaining 18 blocked and 4 rejected statements are recorded honestly as not-yet-verified, not silently omitted, but they are also not verified: any claim about "the corpus" being formally checked must read as "82% of the corpus," not "the corpus."

7.4 The receipts-consolidation gap, twice

Section 4.5 already reports the first consolidation gap: two verified statements silently dropped from a merged receipts file, caught by independent re-verification rather than trusting the consolidated

file's own line count. This thesis document's own authoring process surfaced a second, structurally similar risk: an early draft of the same partition, written as prose in a conversational exchange before this document existed, itself summed to 218 rather than 219 for an unrelated reason (a miscounted category, not the dropped- receipt bug) – caught and corrected before this thesis's own Chapter 4 was drafted. Two independent instances of the same failure mode — a consolidation or restatement step silently dropping or miscounting verified facts — is itself evidence that this failure mode is not a one-off, and that any civilizational-scale version of this pipeline (Chapter 5) needs the independent-reverification discipline built in as a structural requirement, not an incidental good practice.

7.5 The correspondence in Chapter 3 is an analogy, not a proof

Table 3.1's term-for-term mapping between the two μ-pipelines is a structural observation about shared shape (admission gate, defect taxonomy, receipt), not a formal proof that the two pipelines are instances of one algebraic construction in the sense Chapter 2's admission algebra makes precise for a single pipeline. Making that correspondence itself a checked mathematical statement — proving μ-pipeline-hood as a typeclass or trait both implementations satisfy, say — is future work, not something this thesis establishes.

7.6 This document's own manufacturing is new and untested at scale

The thesis document itself (Chapters 1–6) is rendered from RDF paragraph individuals via the `doc-pack` ggen pack, a mechanism built specifically for this document and not previously exercised on a document this size. The `doc:register` field (evidence-bound/speculative) is checked mechanically in Chapter 8's verification step for exactly the paragraphs authored this way; it is a real, novel piece of infrastructure, not a mature, independently-tested tool.

Chapter 8

Conclusion

8.1 What was actually shown

A 219-statement mathematics corpus was routed through an automated pipeline into a disjoint, receipted partition: 179 statements kernel-verified against Lean 4, 4 honestly rejected after bounded retry, 18 correctly withheld on an unresolved dependency, and 17 cross-paper restatements deduplicated before scheduling. This pipeline is a second working instance of a manufacturing morphism already named in this project's own prior work ($\mathcal{A} = \mu(\mathcal{O}^*)$), alongside the existing RDF-to-PDDL pipeline in `src/mfg.rs`. Two real infrastructure bugs and one receipts-consolidation gap were found and fixed during the work, each caught by independent re-verification against the real Lean kernel or the real corpus, never by trusting a self-report.

8.2 What was additionally shown, about this document itself

This document's own Chapters 1–6 are not hand-authored `.tex`: they are rendered by `ggen sync` from RDF paragraph individuals (`docs/thesis/math_manufacturing/rdf/thesis.ttl`) through the `doc-pack` ontology and template, following the same admit/render/receipt shape as the math-manufacturing pipeline it describes. Each paragraph in that data carries a `doc:register` value, `evidence-bound` or `speculative`, and this is a mechanically checkable fact, not a typographic convention: a SPARQL query against the same graph confirms Chapters 1–4 and 6 are entirely `evidence-bound` and Chapter 5 mixes exactly two framing paragraphs with six explicitly `speculative` claims, with no chapter boundary crossed silently. This thesis is therefore, in a narrow and literal sense, an instance of the pattern it argues for: its own structure is admitted data, rendered through a manufacturing morphism, not asserted prose.

8.3 What was not shown

The speculative chapter's central comparison — that the observed throughput on this one corpus generalizes to larger, more diverse corpora, or that a manufacturing-style cost curve for verified mathematics follows from one 59-minute data point — is not shown. It is stated as speculation, marked as such in both the LaTeX environment and the underlying RDF fact, and left as the natural target for future work.

8.4 Closing

The correct adversarial question after this result is not whether gaps exist — Chapter 7 names six of them plainly — but which specific admitted statement, blocked dependency, or evidence-bound claim is wrong, checkable against the receipts, the corpus, and the Lean kernel that produced it.